

Matching de Produtos

T2 - Aprendizado de Máquina

Autores:

João Pedro Antunes,

Gabriel Kaizer,

Vinicius Mibielli

30 de junho de 2026

Sumário

1	Referencial Teórico	2
1.1	NLP Clássico	2
1.2	Deep Learning	2
2	Delimitação do Desenvolvimento	3
3	Metodologia	4
3.1	Pré-Processamento	4
3.2	Pipelines	4
3.3	Avaliação	4
4	Resultados e Análises	5
4.1	Validação	5
4.2	Teste	5
4.3	Tradeoffs	5
4.4	Análise Qualitativa	6
4.5	Consultas sem correspondência (NO_MATCH)	6
5	Conclusão	7
5.1	Melhorias	7
5.2	Dificuldades	7
6	Referências	8

1 Referencial Teórico

O problema abordado é um caso de recuperação de informação: dada uma consulta textual heterogênea (com abreviações, erros e variações de embalagem), deve-se localizar o produto correspondente em um catálogo normalizado. E, para isso, foram empregadas duas técnicas.

1.1 NLP Clássico

TF-IDF (Term Frequency–Inverse Document Frequency) representa cada texto como um vetor esparsos em que o peso de um termo cresce com sua frequência no documento (TF) e diminui quanto mais comum ele é em toda a coleção (IDF). Assim, termos discriminativos (marcas, sabores) recebem peso alto e termos genéricos, peso baixo. A proximidade entre consulta e produto é medida pela similaridade de cosseno, que compara a direção dos vetores e é robusta a diferenças de comprimento entre os textos.

BM25 é uma função de ranqueamento probabilística normalmente usada em motores de busca. Refina o TF-IDF com saturação da frequência de termo (evitando que repetições dominem a pontuação) e normalização pelo comprimento do documento. Costuma ser superior ao TF-IDF em textos curtos.

1.2 Deep Learning

Embeddings semânticos são vetores densos produzidos por redes neurais do tipo transformer que capturam significado, e não apenas correspondência de caracteres. Utilizou-se o modelo multilíngue paraphrase-multilingual-mpnet-base-v2 (Sentence-Transformers), comparando consulta e catálogo novamente por similaridade de cosseno.

LLMs com prompting (zero-shot/few-shot): um modelo de linguagem grande recebe a consulta e uma lista de candidatos e escolhe o mais adequado. Para conter o custo de tokens, os candidatos são filtrados pelo BM25 (top 10) e só então enviados ao LLM (Gemini ou Claude). Sem exemplos no prompt, tem-se o cenário zero-shot, com exemplos de correspondências corretas, few-shot.

2 Delimitação do Desenvolvimento

Reaproveitado: scikit-learn (TfidfVectorizer, cosine_similarity), rank-bm25 (BM25Okapi), sentence-transformers (modelo de embeddings pré-treinado), nltk (lista de stopwords em português), API do Google Gemini (google-genai) e API do Anthropic Claude (anthropic).

Implementado pelo grupo: normalização textual de domínio com a função normalize (expansão de abreviações, remoção de indicadores de embalagem, tratamento de S/), os pipelines de busca (search_tfidf, search_bm25, search_embeddings), a construção dos prompts zero-shot/few-shot e a lógica de re-ranqueamento via LLM, as métricas (P@1, MRR@5 e R@5), o cache de embeddings e o script parse_log.py (recalcula as métricas do Gemini a partir do log, sem repetir as chamadas à API).

3 Metodologia

3.1 Pré-Processamento

Aplicado tanto às consultas quanto aos nomes do catálogo:

- Conversão para minúsculas,
- padronização de unidades (350 ml → 350ml, 2 litros → 2l),
- expansão de s/ → "sem",
- remoção de acentos (normalização NFKD),
- remoção de indicadores de embalagem (c/6, und),
- remoção de pontuação,
- stopwords em português (NLTK).

3.2 Pipelines

Abordagem 1 (NLP Clássico): vetorização TF-IDF e indexação BM25 do catálogo. Para cada consulta, calcula-se a similaridade com todos os produtos e retornam-se os top-5, registrando o `product_id` do 1º colocado como predição principal.

Abordagem 2 (Deep Learning): embeddings semânticos, vetores densos da consulta e do catálogo, com similaridade de cosseno e mesmo formato de saída (top 5), onde os embeddings do catálogo são armazenados em cache (.npy) para acelerar as execuções; e re-ranqueamento com LLM, onde os top 10 candidatos do BM25 são reordenados pelo Gemini/Claude (zero-shot e few-shot).

3.3 Avaliação

Os ajustes e a comparação entre abordagens foram feitos no conjunto de validação (`queries_val.csv`, 250 consultas). O conjunto de teste (`queries_test.csv`, 250 consultas) foi usado apenas ao final, para as métricas definitivas. As três métricas pedidas, P@1, MRR@5 e R@5 foram implementadas diretamente.

4 Resultados e Análises

4.1 Validação

Abordagem	P@1	MRR@5	R@5	Tempo(s)	Custo
TF-IDF + cosseno	0,960	0,979	1,000	2	Gratuito
BM25	0,956	0,976	1,000	2	Gratuito
Embeddings (mpnet)	0,788	0,841	0,924	10	Gratuito
BM25 + Gemini (zero-shot)	0,968	0,983	1,000	27 min	API
BM25 + Claude (few-shot)	0,968	0,983	1,000	5 min	API

Tabela 1: Tabela de resultados da Validação.

4.2 Teste

Abordagem	P@1	MRR@5	R@5	Tempo(s)	Custo
TF-IDF + cosseno	0,976	0,986	1,000	2	Gratuito
BM25	0,972	0,984	1,000	2	Gratuito
Embeddings (mpnet)	0,844	0,878	0,928	12	Gratuito
BM25 + Gemini (zero-shot)	0,972	0,985	1,000	43 min	API
BM25 + Claude (few-shot)	0,972	0,986	1,000	5 min	API

Tabela 2: Tabela de resultados do Teste.

4.3 Tradeoffs

O principal resultado é que as abordagens clássicas superaram os embeddings puros. O matching de produtos é fortemente lexical, onde a marca, sabor e tamanho precisam coincidir exatamente. O TF-IDF e o BM25 capturam bem esses tokens discriminativos, enquanto os embeddings tendem a aproximar variantes semanticamente parecidas, mas que correspondem a produtos diferentes (ex: "tradição" e "extraforte", ou 900ml e 1,4l).

O TF-IDF foi o melhor método puramente local ($P@1 = 0,960$ na validação e $0,976$ no teste), rápido e sem dependências pesadas. O re-ranqueamento por LLM sobre os candidatos do BM25 elevou ligeiramente o desempenho na validação ($P@1 = 0,968$), mostrando capacidade de corrigir casos em que o produto correto não ocupa a primeira posição do BM25. Entretanto, esse ganho não se manteve no conjunto de teste, em que o TF-IDF obteve o maior $P@1$. Assim, o principal tradeoff do uso de LLMs é um pequeno ganho potencial de precisão em alguns cenários, ao custo de elevada latência, dependência de API e limites de uso, sem vantagem consistente em relação ao TF-IDF nos experimentos realizados.

4.4 Análise Qualitativa

- Acertos, mesmo com reordenação de palavras:
 - **Consulta:** *Café solúvel tradição forte granulado nescafé sachê 40g*
 - **Correto:** *"Café Solúvel Granulado Forte Nescafé Tradição Sachê 40g"*
- Erros dos embeddings, que o NLP Clássico acerta (confusão de variante/tamanho):
 - **Consulta:** *"Lava-Roupas Líquido Lavanda Omo Sachê 900ml Refil"*
 - **Correto:** *Lava roupas líquido omo lavanda sachê refil 900ml*
 - **Embedding:** *Lava roupas líquido omo ciclo rápido antiodor 1,4 litros*
- Erros do BM25 (sobreposição enganosa de tokens e confusão de marca):
 - *"Ketchup Heinz 1,033kg" → previu "Salgadinho Doritos ketchup Heinz 110g"*
 - *"Manteiga Extra sem Sal Aviação 200g" → previu "...Batavo 200g"*
- **Casos ambíguos:** o catálogo contém produtos com nomes idênticos, mas EANs diferentes. Ex: *"Molho shoyu light sakura 150ml"* existe com dois `product_id` distintos, fazendo com que o sistema retorne o nome certo, porém com a possibilidade de escolher o EAN "errado", sendo penalizado mesmo estando correto.

4.5 Consultas sem correspondência (NO_MATCH)

Tratamento para identificar quando uma consulta não possui nenhum produto correspondente no catálogo. Para avaliação, simulamos consultas sem correspondências removendo do catálogo o produto correto ou toda a categoria relacionada a ele e verificamos qual é o maior score de similaridade (TF-IDF) obtido pela consulta.

O resultado da simulação mostra dois cenários:

- **1. Produto muito parecido ainda existe no catálogo**
 - Exemplo: a consulta é "Coca 1L", mas apenas "Coca 2L" está disponível
 - O score de similaridade continua alto ($\approx 0,74$), pois o sistema acredita que encontrou um bom resultado, mesmo que o produto não exista
- **2. Produto realmente não existe no catálogo**
 - Exemplo: nenhuma Coca-Cola está cadastrada
 - O score cai para cerca de 0,33, tornando mais fácil detectar que não há correspondência

O limiar moderado adotado foi de 0,30 sobre o cosseno do TF-IDF, onde abaixo dele a consulta é marcada como NO_MATCH. Esse valor preserva 100% dos matches corretos e detecta casos onde o produto realmente está fora do catálogo. Porém, o método ainda não consegue distinguir quando existe um produto muito parecido. Para esse casos, seria necessário usar informações adicionais, como comparar marca, volume ou deixar que um modelo de linguagem (LLM) decida quando deve se abster de responder.

5 Conclusão

As abordagens clássicas mostraram-se fortes e baratas para este problema lexical; os embeddings puros, sozinhos, ficaram atrás; e o re-ranqueamento por LLM ofereceu o melhor desempenho, ao custo de latência e dependência de API.

Para este problema, as abordagens clássicas bem pré-processadas são a melhor escolha: TF-IDF e BM25 atingiram $P@1 \approx 0,95$ e $R@5 = 1,000$, de forma gratuita e quase instantânea, igualando o re-ranqueamento por LLM. Uma das correções mais impactantes foi de também normalizar o catálogo, e não apenas as consultas.

5.1 Melhorias

- Re-codificar os embeddings com o catálogo normalizado e avaliar se a abordagem semântica se aproxima das clássicas (versão atual usa o catálogo original em cache).
- Desambiguar EANs com nome idêntico usando alguma coluna.
- Adicionar o LLM apenas nos casos incertos (empate de score entre os primeiros candidatos), reduzindo chamadas de API, e combinar score + margem (top 1 - top 2) para melhorar a detecção de NO_MATCH em produtos quase duplicados.

5.2 Dificuldades

- Ruído e abreviações nas consultas;
- nomes duplicados no catálogo;
- limites de rate-limit da API grátis do Gemini, contornados com pausas automáticas e com o cache de log para reavaliação offline.

6 Referências

- <https://machinelearningmastery.com/build-semantic-search-with-llm-embeddings/>
- Documentação: scikit-learn, rank-bm25, Sentence-Transformers, Google Gemini API e Anthropic Claude API.